Release: 20260320

Welcome

Welcome to **AI & Vector Search Basics course**!

Tell us about yourself

- Your name
- Your role within your company
- What is your experience with Vectors and Machine Learning?
- What would you like to get out of this course?

Topics we cover

Introduction
Setup
Intro to Vectors and Vector Search
Approximate Nearest Neighbors (ANN)
Hierarchical Navigable Small Worlds (HNSW)
Text Embeddings and Chunking Techniques
Using Atlas Vector Search
MongoDB's applications as a Vector Database
Atlas Vector Nodes

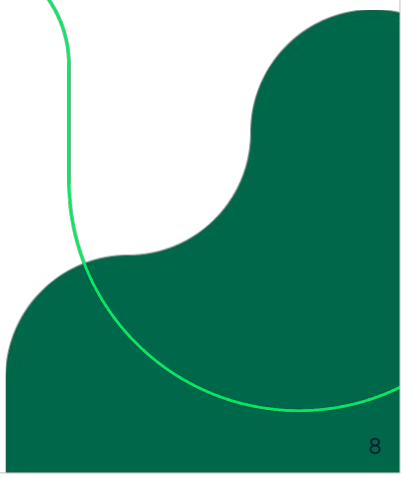
Introduction

This course introduces Vector Search, the relevant basic concepts, and focusing on semantic search using Atlas Vector Search.

Some of the key concepts you will learn today include:

- Vectors in the context of AI
- ANN & HNSW algorithms
- Text embeddings
- Chunking techniques
- Atlas Vector Search
- Atlas Vector Nodes

Setup



Setup

This course will use an Atlas Cluster and the sample dataset:

1. Create an Atlas account
2. Create an M0 (or higher) Atlas cluster
3. Load sample dataset

In this course we will use Python code examples.
The course lab will have these already installed.

Some exercises use our MDB Azure instance for getting embeddings using OpenAI's ***text-embedding-ada-002*** model.
The endpoint provided for requesting the embeddings to the server will require an API KEY.

Creating an Atlas cluster : https://www.mongodb.com/docs/atlas/tutorial/create-new-cluster/
Loading sample data set: https://www.mongodb.com/docs/atlas/sample-data/

Note: Atlas M0 cluster has some limitations regarding storage and a maximum number of 3 atlas search indexes.
If you choose to use an M0 cluster be aware you will need to delete an index you created in previous exercises to complete all the course exercises.
The API Key for our MDB Azure instance will be provided by the training team.

Intro to Vectors

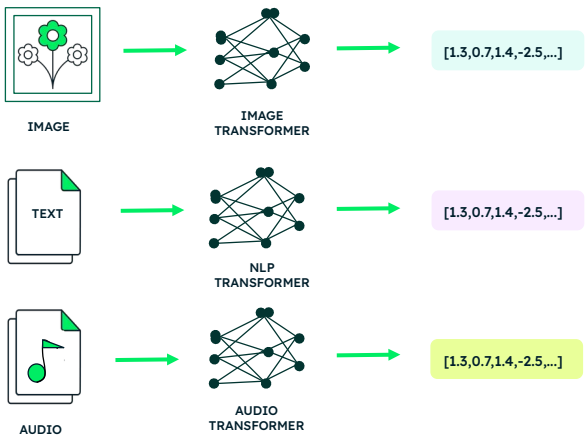
Vector Applications in AI

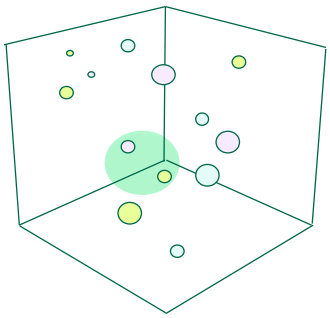
Vectors play a significant role in various AI applications:

- Word Embeddings
- Semantic Search
- Generative AI

Vectors in the context of AI

Vectors are a series of numbers arranged in a specific order and used to represent data.





Vectors are mathematical objects that are commonly used to represent and manipulate data.
They are often used to represent features or attributes of an object or a data point.
Each number in the array represents a certain dimension or property of the object being represented.
Representations of data (like text, images, videos, users, music, etc) which are *similar* will occupy points in N-dimensional space *close together*.

Vectors in the context of AI

Example

Classification (meaning)

Data

	Object	Creature	Female	Male	Young	Human	Feline	Reptile
cat	0	1	1	1	0	0	1	0
dog	0	1	1	1	0	0	0	0
table	1	0	0	0	0	0	0	0
chairs	1	0	0	0	0	0	0	0
woman	0	1	1	0	0	1	0	0
men	0	1	0	1	0	1	0	0
girl	0	1	1	0	1	1	0	0
snakes	0	1	1	1	0	0	0	1

Imagine we have some words that we would like to represent as vectors: cat, dog, table, chairs, woman, men, girl, snakes.
If we encode these words as vectors based on this 8-category representation, the word "cat" is embedded as [0,1,1,1,0,0,1,0]

Vectors in the context of AI

Example

Classification (meaning)

Data

	Object	Creature	Female	Male	Young	Human	Feline	Reptile
cat	0	1	1	1	0	0	1	0
dog	0	1	1	1	0	0	0	0
table	1	0	0	0	0	0	0	0
chairs	1	0	0	0	0	0	0	0
woman	0	1	1	0	0	1	0	0
men	0	1	0	1	0	1	0	0
girl	0	1	1	0	1	1	0	0
snakes	0	1	1	1	0	0	0	1
tiger	0	1	1	1	0	0	1	0

If we use the same embedding model to encode the word "*tiger*", we would get a similar vector embedding: [0,1,1,1,0,0,1,0]. Comparing the vectors allows us to see the similarities between words.
Try to practice with the same encoding model using the word: "boys". What similarities do you get?
Because we only have 8 categories, and each category can only contain a boolean value, we are limited in the amount of information we can represent in each vector.

Vectors in the context of AI

Example

Classification (meaning)

Data

	Object	Creature	Female	Male	Young	Human	Feline	Reptile
cat	-0.78	0.79	0.15	0.20	0.75	-0.68	0.97	-0.5
dog	-0.69	0.81	0.22	0.36	0.45	-0.34	0.02	-0.67
table	0.79	-0.82	0.28	-0.13	-0.01	-0.98	-0.77	-0.75
chairs	0.88	-0.87	0.14	-0.05	-0.08	-0.86	-0.80	-0.82
woman	-0.66	0.80	0.91	-0.11	-0.12	0.99	-0.57	-0.45
men	-0.64	0.85	-0.25	0.88	-0.23	0.98	-0.47	-0.75
girl	-0.66	0.80	0.91	-0.11	0.83	0.87	-0.57	-0.45
snakes	-0.5	0.71	0.01	0.46	-0.03	-0.54	-0.12	0.96
tiger	-0.68	0.80	0.23	0.21	0.57	-0.72	0.98	-0.65

Instead of discrete values, we could use a continuous scale from -1 to 1 representing the *probability* that we can classify the word in the category. This allows us to capture finer details.

Word Embeddings

Represent words/text as high-dimensional vectors:

CAT

"embedding": [
-0.0070945825, -0.01732811, -0.0096440865, -0.030707682, -0.012548676, 0.0031052113,
-0.0051132124, -0.041218173, -0.014629469, -0.02137607, 0.01923136, 0.050876465,
-0.001290731, 0.0024855894, -0.038405906, -0.006089694, 0.03550842, -0.004697764,
0.0023630853, -0.013429284, -0.018918887, 0.009019138, 0.01589357, -0.008713766,
-0.014672079, ..., -0.014366707, -0.023435557, -0.014167859]

TIGER

"embedding": [
-0.020038323, -0.017262567, -0.013343462, -0.009503664, -0.019787183, 0.021214716,
-0.03172294, -0.0061958865, -0.006800605, -0.026739795, 0.0007121139, 0.0034135205,
0.010706492, -0.0055581233, -0.012517343, -0.011651572, 0.029396592, -0.0016728895,
0.023897948, 0.004870793, -0.007091399, 0.015319536, 0.0106734475, 0.0017926765,
-0.0024287875, ..., -0.023805423, 0.018148165, -0.0026055768]

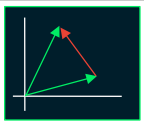
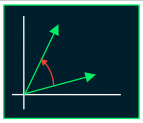
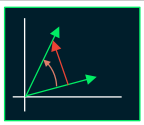
Word embeddings
In natural language processing (NLP), words are often represented as high-dimensional vectors called word embeddings.
These vectors capture the semantic meaning of words, allowing algorithms to understand and process text data more effectively.

Examples: The word CAT and the word TIGER were encoded as vectors/embeddings using OpenAI embedded model ***text-embedding-ada-002***

Vector Similarity Functions

We can find semantic similarities by calculating vector similarities.

Atlas Vector Search supports 3 different similarity functions:

Euclidean	Cosine	Dot Product
Measures Distance  $\sqrt{(a_1-b_1)^2+(a_2-b_2)^2+\ldots+(a_n-b_n)^2}$	Measures Angle  $\frac{a^t b}{\lvert a\rvert\cdot\lvert b\rvert}$	Measures Magnitude + Angle  $a_1b_1 + a_2b_2 + \ldots a_nb_n = \lvert a\rvert\lvert b\rvert cos(\theta)$

17

Semantic search
By representing text documents and search queries as vectors, we can measure the similarity between them using vector operations like the dot product or cosine similarity.
This enables us to perform a semantic search, where the search engine returns results based on the meaning of the query rather than just keyword matching.

- **Euclidean** is good for cases where we want to find similar data, which has been encoded to occupy a specific region in vector space.
- **Cosine** is good for identifying outliers, which would have a large angular distance from other data, regardless of magnitude.
- **Dot product** is good for recommendation systems, where similar items would have a small angular difference, but magnitude matters.

Types of Vectors

Sparse

```
[0, 0, 0, 0, 0, 0.48, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

Dense

```
[0.12, -0.45, 0.67, 0.89, -0.34, 0.56, -0.71, 0.29, 0.49, -0.21]
```


Sparse vectors are high-dimensional vectors with a majority of their elements being zero.
Dense vectors are vectors with most of their elements being non-zero. They are more "packed" with information compared to sparse vectors.

Examples of scenarios where these type of vectors are important in Generative AI

- In semantic search, documents and queries are often represented as high-dimensional vectors.
  - Sparse vectors can be used to represent term frequency-inverse document frequency (TF-IDF) weights.
  - Dense vectors can be used for word embeddings or document embeddings generated by neural networks.
- In generative models,
  - Dense vectors capture the meaning of words and phrases, which the model uses to generate text that is coherent and contextually relevant.

Types of Vectors

Sparse	Dense
High proportion of elements with zero values	Most elements have non-zero values
Can be represented more efficiently in memory using data structures like dictionaries or compressed sparse row (CSR) format, which only store non-zero elements	Require more memory as they store all elements
Operations can be faster due to the presence of fewer non-zero elements	Some algorithms and data structures are optimized for dense vectors, making them more efficient in certain cases

Approximate Nearest Neighbours (ANN) Search

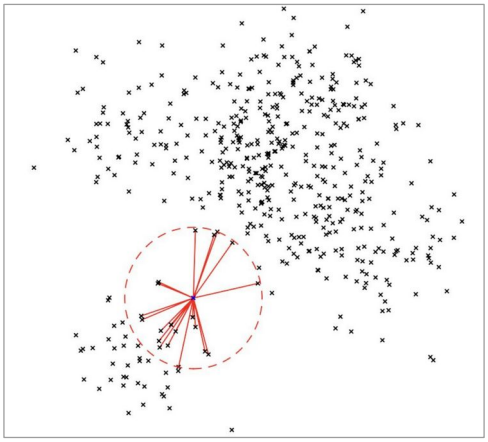
Approximate Nearest Neighbours (ANN)

ANN is a technique used in the field of AI and data mining.

Finds approximate nearest neighbours to a given query point in a large dataset.

Atlas Vector Search performs ANN searches on vector fields

ENN (Exact Nearest Neighbor) is also an option for specific cases

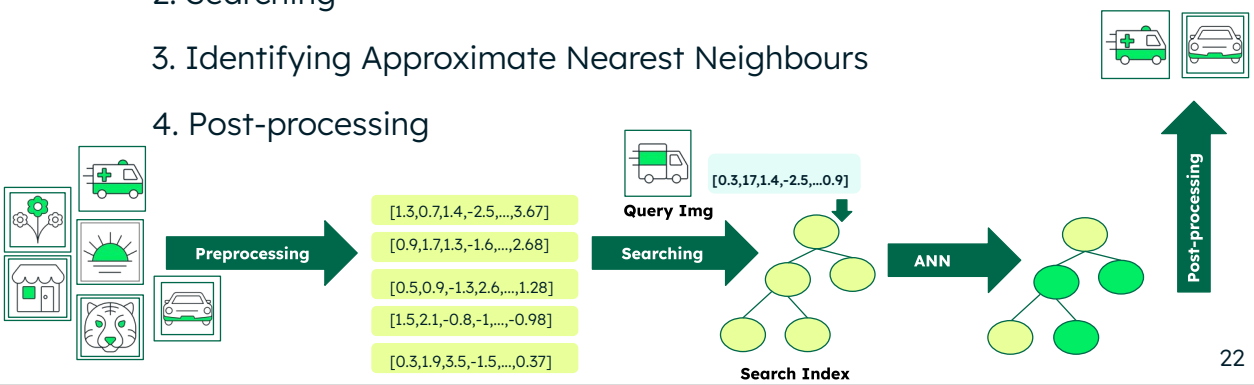


The accuracy of the approximate nearest neighbour search depends on the specific algorithm and parameters chosen.
The trade-off usually lies between efficiency and accuracy, meaning that higher accuracy comes at the cost of longer search times.
Exact Nearest Neighbor (ENN) is also an available option when accuracy is important.

How does ANN work?

An ANN search goes through the following steps:

1. Preprocessing
2. Searching
3. Identifying Approximate Nearest Neighbours
4. Post-processing



To understand how ANN search works, let's consider the example of a large dataset of images, and you want to find the similar images to a particular query image.

1. **Preprocessing:** In the preprocessing step, the dataset is transformed into a more suitable format for efficient searching. This typically involves converting the images into a feature representation that captures their characteristics, i.e. into vectors.

2. **Searching:** When a query image is given, the search algorithm starts from the root of the search index and traverses, comparing the query image's features with the features of the data points at each node. Based on the similarities, it decides which step to go to next. This process continues until it reaches the last item in the data set or a certain stopping criterion is met.

3. **Approximate Nearest Neighbours:** While searching, a set of potential nearest neighbour candidates is obtained. These candidates may not be the exact nearest neighbours, but they are close enough to be considered.

4.Finally, a **post-processing** step is performed on these candidates to further refine the search and find the closest matches to the query image.

Follow this exercise in the Student workbook.

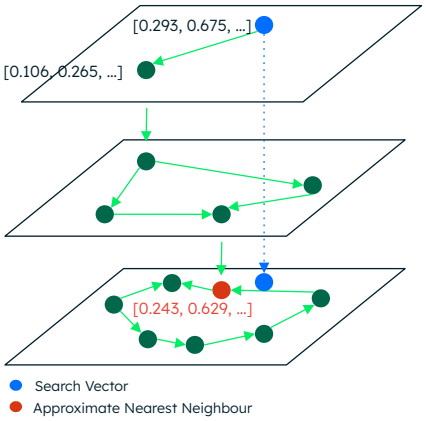
Hierarchical Navigable Small Worlds (HNSW) Algorithm

Hierarchical Navigable Small Worlds (HNSW)

HNSW builds a hierarchical structure that allows for efficient nearest neighbour searches.

High layers have less points for faster searches. Lower layers have more points for accurate searches.

Searches complete after a specified number of candidates have been compared.



HNSW: Hierarchical Navigable Small World

The Hierarchical Navigable Small Worlds (HNSW) algorithm is an efficient and scalable approach for approximate nearest neighbour (ANN) search in high-dimensional spaces.

Atlas Vector Search uses the HNSW algorithm to index and search for Vectors.

During its construction phase, the HNSW algorithm starts by randomly selecting a vector to be the *first* node in the graph. Then, it iteratively adds new vectors to the graph, connecting them to existing nodes based on their similarity.

Once the graph is constructed, it provides an efficient way to search for approximate nearest neighbours. We can specify the "Number of Candidates" we are willing to consider. This allows us to trade off speed versus accuracy.

Why is HNSW important?

Efficient Search:
Designed to perform fast approximate nearest neighbour search.

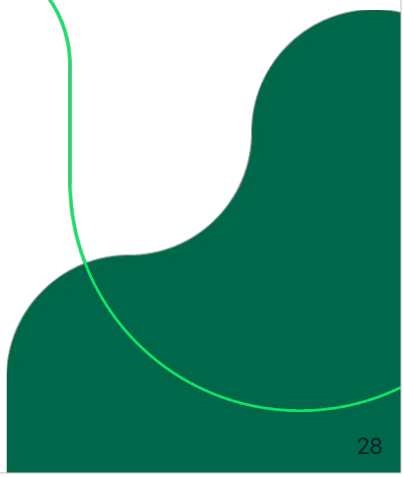
High Recall:
Has a high recall rate, it can find the most similar vectors with high accuracy.

Approximate Search:
Offers approximate search capabilities, meaning that it sacrifices a small degree of accuracy in favor of significantly faster search times.

Versatility:
Can be applied to a wide range of applications that require vector search.

Overall, HNSW is an important algorithm in vector search as it provides efficient, scalable, and accurate approximate nearest neighbour search, making it a valuable tool in various AI applications.
Atlas Vector Search uses the Hierarchical Navigable Small Worlds (HNSW) algorithm to perform searches. This allows Atlas Vector Search to find the most similar documents to a given query document, even if the documents are not in the same order in the database. This makes it ideal for semantic search tasks, such as finding the most similar documents to a given query document.

Text Embeddings & Chunking Techniques

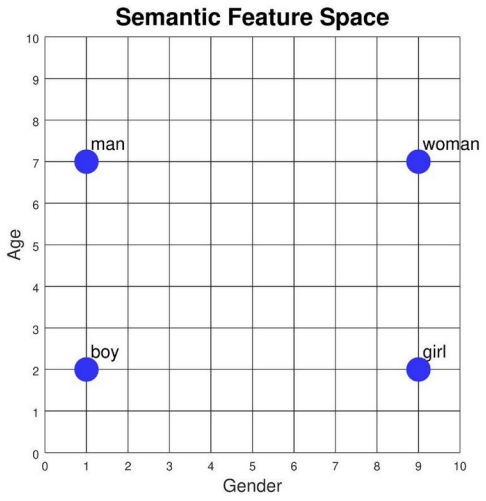


Text Embeddings

Word embeddings or word vectors represent words or sentences or paragraphs as numerical representations.

These representations capture the semantic and contextual information of the text (or any other unstructured data).

They are used in natural language processing (NLP) and machine learning



Text embeddings or word embeddings allow machines to work with and understand human language more effectively.
This allows us to perform mathematical operations on these vectors, such as calculating the similarity between two words or finding the most similar words to a given word.
https://www.cs.cmu.edu/~dst/WordEmbeddingDemo/tutorial.html

Models to generate Embeddings

Some of the most popular embedding models are:

- Word2Vec
- GloVe
- FasText
- BERT

In this course we will use OpenAI's: "text-embedding-ada-002" and "clip-ViT-L-14" embeddings

Word2Vec
Google's Word2Vec uses a shallow neural network to learn the vector representations of words in a large corpus of text.

GloVe
Short for Global Vectors for Word Representation, GloVe is an unsupervised learning algorithm developed by Stanford University. It combines the benefits of both global matrix factorization and local context window methods to generate word embeddings.

FastText
Facebook's FastText is an extension of Word2Vec that can generate embeddings for words and subwords. This allows it to handle out-of-vocabulary words and morphologically rich languages more effectively.

BERT
Bidirectional Encoder Representations from Transformers (BERT) is a pre-trained deep learning model developed by Google. It generates contextualized word embeddings by considering the context of a word within a sentence, making it more powerful than traditional word embeddings.

Semantic search with Text Embeddings

By using text embeddings, we can perform tasks such as:

- Finding the most relevant documents for a given query
- Identifying similar documents or sentences
- Clustering documents based on their semantic content
- Detecting and correcting spelling errors in queries

Vector Embedding Indexing

Similar to the concept in traditional DBs:

Allows quicker access to data

Similar vectors are stored next to each other

Enables fast proximity or similarity searches

Text Embedding Example

Run this HTTP request in the Terminal:

```
curl https://iltvectorsearch.openai.azure.com/openai/deployments/text-embedding-ada-002/embeddings?api-version=2023-05-15 \
-H 'Content-Type: application/json' \
-H "api-key: $AZURE_OPENAI_API_KEY" \
-d '{"input": "Sample Document goes here"}'
```

Review the response, the input embedding is included.

Change the input and test again.


```
{
  "object": "list",
  "data": [
    {
      "object": "embedding",
      "index": 0,
      "embedding": [
        -0.020038323,
        -0.017262567,
        -0.013343462,
        -0.009503664,
        -0.019787183,
        0.021214716,
        -0.03172294,
...
```


In order to understand better, let's see what embeddings look like.
Ensure that you have set the $AZURE_OPENAI_API_KEY environment variable variable with the AI API Key as described in the workbook: Section 2.3 Getting the AI access token

Text Embedding Usage

Applications	Challenges
Most common use cases: Search engines Recommender systems Sentiment analysis Machine translation	Computational complexity Handling ambiguity Bias

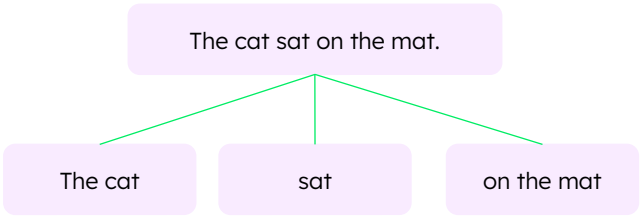
Text embeddings have a wide range of applications in natural language processing, information retrieval, and machine learning, but these also come with challenges.

Chunking

Computational method used in the field of AI and NLP.

Process of dividing a sequence of words or tokens into meaningful **chunks**.

Chunks typically consist of a group of words that form a syntactically coherent and semantically important unit.



In natural language processing, chunking is used to identify and extract specific parts of speech or phrases from a sentence. This can include noun phrases, verb phrases, prepositional phrases, and more. By breaking down a sentence into chunks, it becomes easier to analyze and understand the underlying structure and meaning of the text.

Why is Chunking important?

Enables interpretation, extraction, and understanding of meaning from unstructured data

Different applications:

- Parsing text
- Named entity recognition
- Information extraction
- Machine learning (ML) and natural language processing

Chunking is important in AI because it enables the interpretation, extraction, and understanding of meaningful information from unstructured data, contributing to various applications like text parsing, named entity recognition, information extraction, and natural language processing.

Follow this exercise in the Student workbook.

Chunking Techniques

Most common chunking techniques used in NLP are:

- Word-level
- Sentence-level
- Paragraph-level

Paragraph	Sentence	Word
One of the steps to be performed in the NLP. It converts unstructured text into a proper format of data.	One of the steps to be performed in the NLP. It converts unstructured text into a proper format of data.	One of the steps to be performed in the NLP it converts text unstructured into of a proper format data

Chunks can be individual words, sentences, or even paragraphs. The goal of chunking is to extract meaningful information from text (and still have enough context to make the chunk meaningful for the purpose intended), which can then be used for various tasks like sentiment analysis, named entity recognition, or text classification.

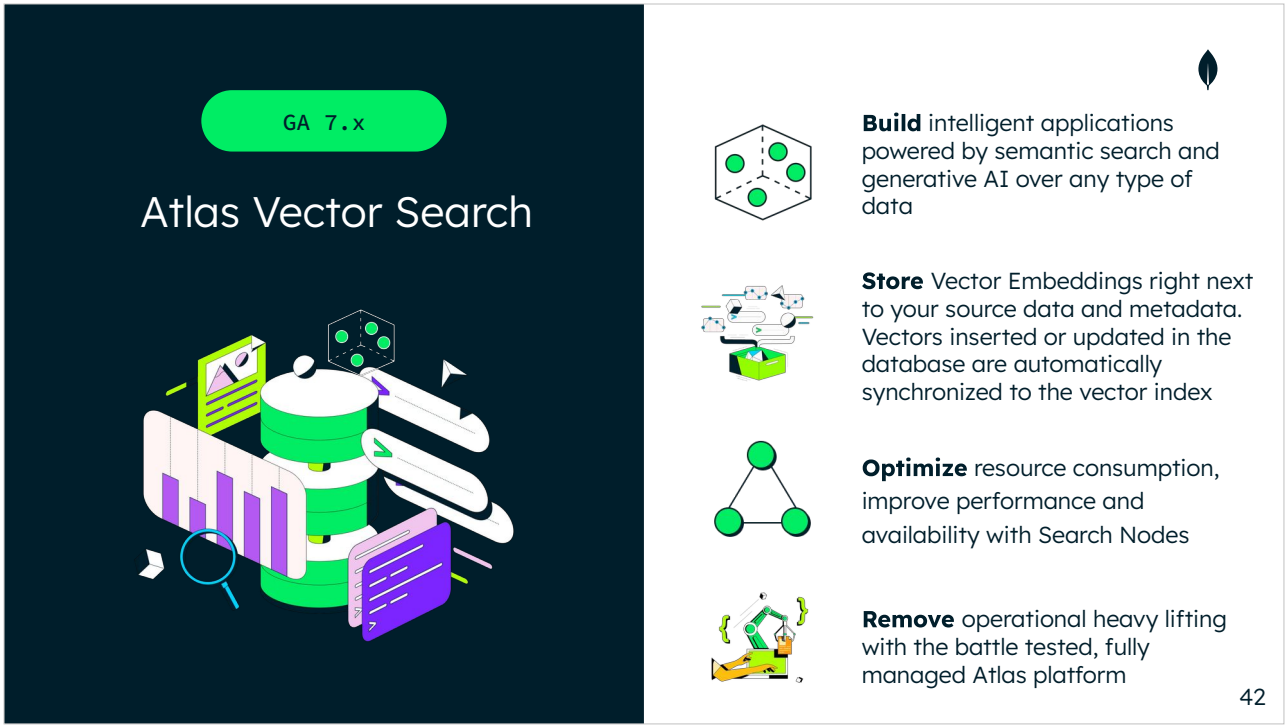
Chunking Discussion

Discuss the most appropriate chunking technique based on these use cases:

Use Case	Suitable Chunking Technique
Sentiment analysis	Word Level
Information extraction from news articles	
Categorizing documents	
Text summarization	

Follow this exercise in the Student workbook.

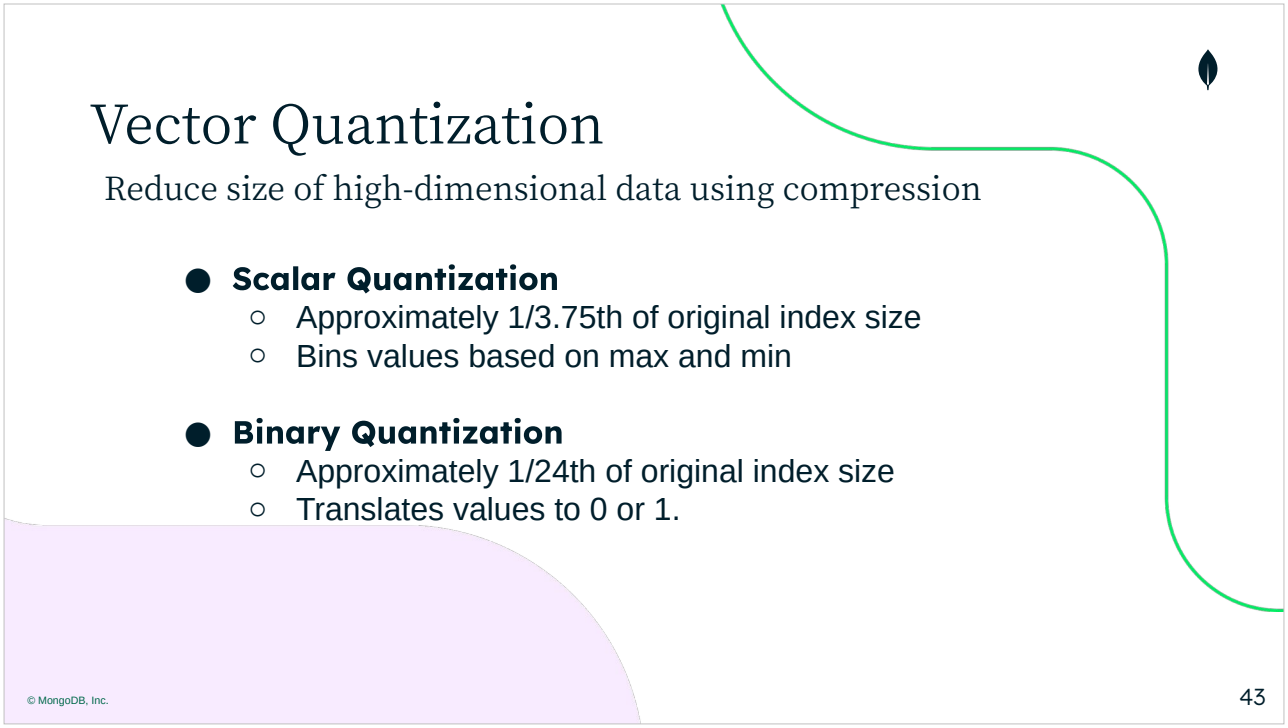
Atlas Vector Search



Atlas Vector Search is part of Atlas and Atlas Search. We've designed it on our Atlas Search platform, which uses Lucene, an engine built with semantic search and generative AI in mind. Since Atlas Search is part of MongoDB Atlas, it leverages the power of MongoDB for vector storage right next to the data's source. Vectors inserted and updated in MongoDB are automatically synchronized to the vector index.

In Atlas, you can provision Search Nodes, dedicated hardware that allows for optimum compute resources and better performance and availability for vector search use cases.

Atlas removes the operational heavy lifting of keeping the data's source of truth in sync with the search database using the Mongodb Cloud Platform.



Vector quantization is a data compression technique that approximates high-dimensional vectors with a smaller, finite set of representative vectors.
This approach reduces the memory footprint of embeddings, which lowers infrastructure costs and enables faster similarity searches.
Quantization improves scalability, though it come with an expected reduction in accuracy.

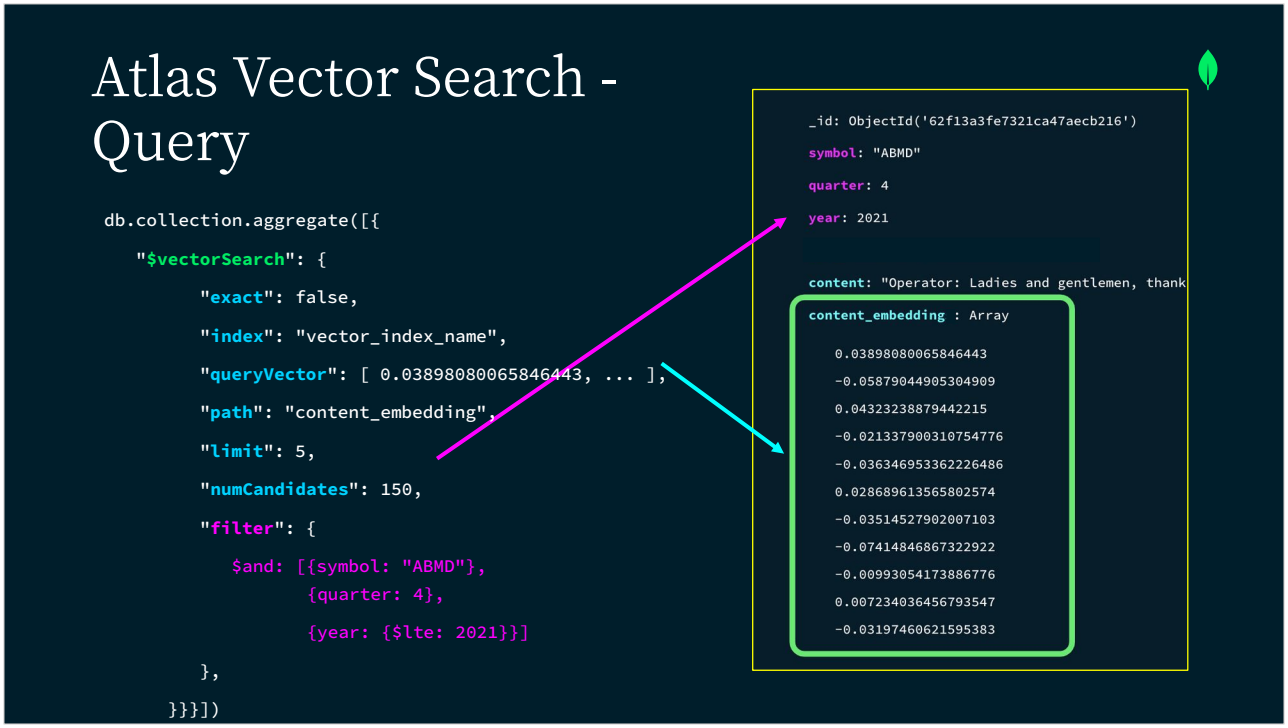
- Scalar quantization:
  - Identifies the maximum and minimum values for each dimension of the indexed vectors to establish a range of values for the given dimension. This range is then divided into equally sized intervals or bins. Each float value is then mapped to a bin, converting the continuous value into a discrete integer.
  - In Atlas Vector Search, this quantization reduces the vector embedding's RAM cost to about one fourth (1 / 3.75) of the pre-quantized cost.

- Binary quantization:
  - Assumes a midpoint of 0 for each dimension. Then each value in the vector is compared to the midpoint and assigned a binary value of 1 if it's greater, or 0 if it's less than the midpoint value.
  - In Atlas Vector Search, this quantization reduces the RAM cost of the vector embedding to one twenty-fourth (1/24) of the pre-quantization cost.
  - The reason it's not 1/32 is because the data structure containing the

Atlas Vector Search - Index Definition

```
{
  "fields": [{
    "path": "content_embedding",
    "type": "vector",
    "dimensions": 1536,
    "similarity": "<euclidean | dotProduct | cosine>",
    "quantization": "<binary | scalar>",
    },
    {
     "type": "filter",
     "path": "symbol"
    },
    {
     "type": "filter",
     "path": "year"
    }
  ]
}
```


You need to create the vector index in Atlas in order to use the Atlas Vector Search.
Vector index has to be defined based on this syntax. The type is vector.
The path defines the field containing the embedding in our collection
The number of dimensions are based on the embeddings.
The similarity is the function that will be used to find similarities.
The quantization is the method that will be used to compress the vectors.
Additional filters can also be defined as optional parameters.



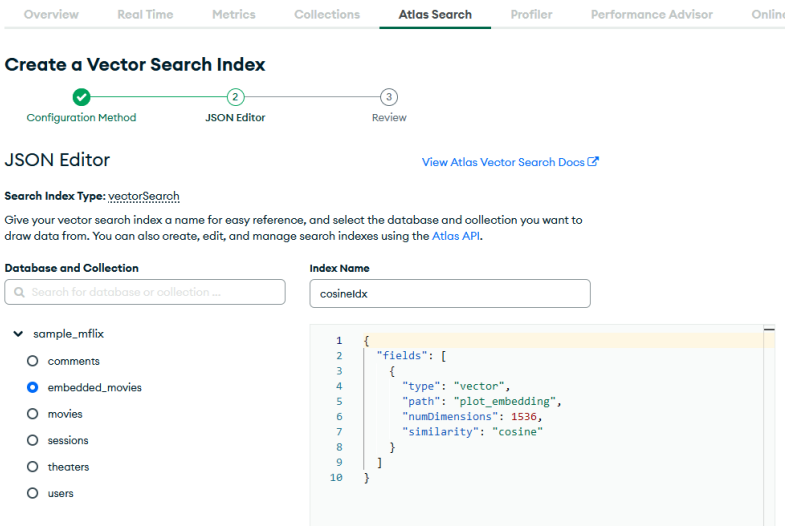
When using the Vector search, you define an aggregation pipeline with the $vectorSearch stage.

This stage includes the vector index to be used, the query vector, and the path as the field in the collection containing the embeddings.
The query vector must be the result of generating an embedding using the same model which generated the embedding in the 'path' field.

Additionally we can apply filters on **boolean**, **numeric** and **string** values (dates or timestamps are not supported). Filtering occurs *before* the vector search is run.
You must define the fields you will filter by when you define the Vector Search index.

limit specifies the number of documents to return
numCandidates specifies the number of documents to consider.
exact is a boolean that indicates if the vector search is using ANN Approximate (false) or ENN Exact (true). If omitted, it is false (ANN) by default,

Creating the Atlas Vector Search index



Reviewing embedded_movies collection

Using Atlas, inspect the **sample_mflix.embedded_movies** collection

```
use sample_mflix
db.embedded_movies.findOne()
```

Identify which field contains the embeddings of the movie plot

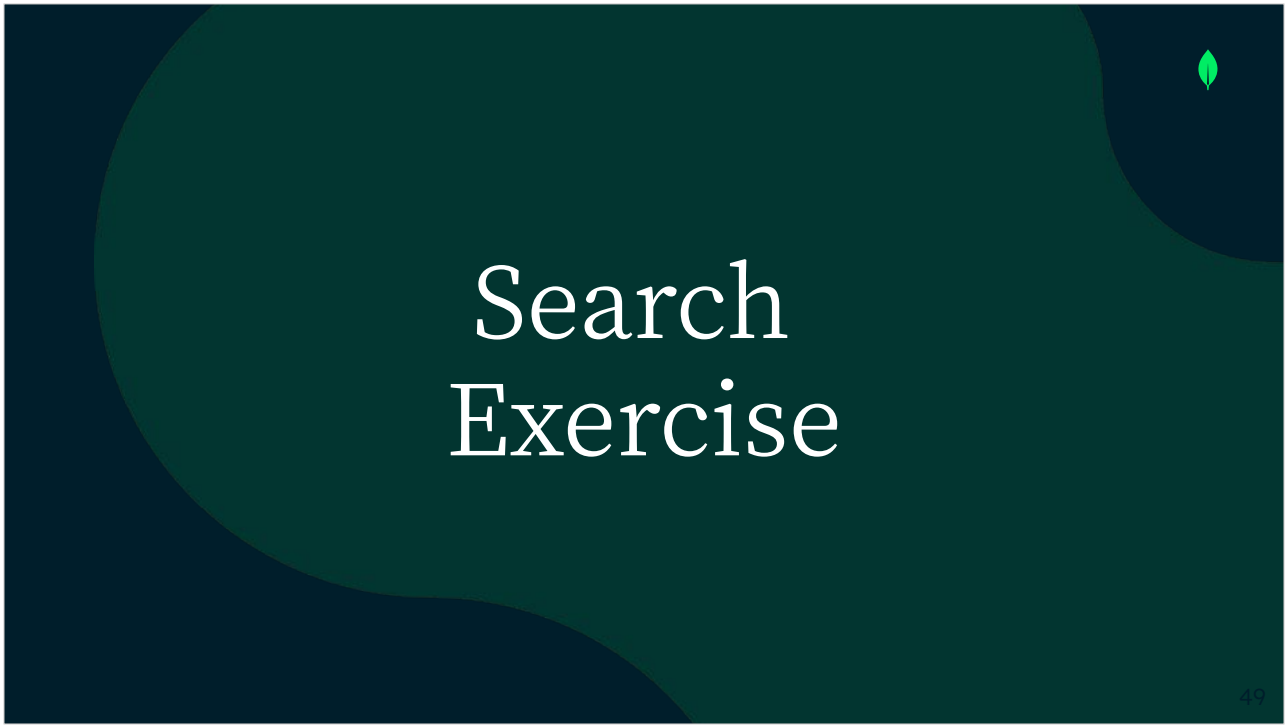
Use the MDB documentation and find details about the embedding model used to create the field.
Review https://www.mongodb.com/docs/atlas/sample-data/sample-mflix/#std-label-mflix-embedded_movies

From the sample data set loaded in the cluster, we'll be using the **sample_mflix.embedded_movies** namespace. The **embedded_movies** collection already stores embeddings in the **plot_embeddings** field, created using the plot field.

Creating embedded_movies indexes

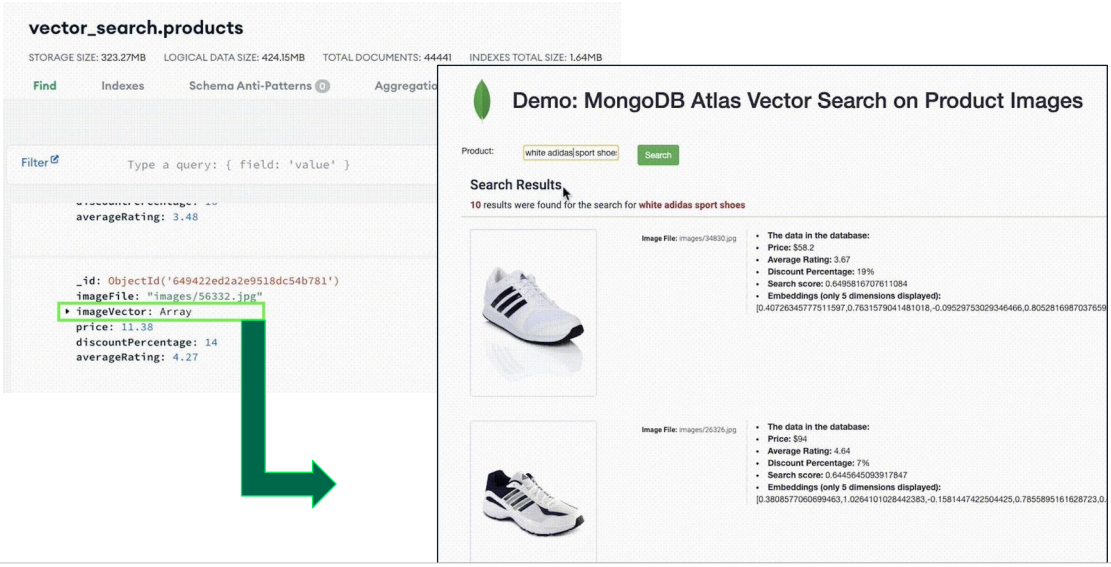
Create 3 different Atlas Vector Search indexes based on the ***plot_embedding*** field of the ***embedded_movies*** collection:

cosineIdx	euclideanIdx	dotProductIdx
`{ "fields": [ { "type": "vector", "path": "plot_embedding", "numDimensions": 1536, "similarity": "cosine" } ] }`	`{ "fields": [ { "type": "vector", "path": "plot_embedding", "numDimensions": 1536, "similarity": "euclidean" } ] }`	`{ "fields": [ { "type": "vector", "path": "plot_embedding", "numDimensions": 1536, "similarity": "dotProduct" } ] }`



Follow this exercise in the Student workbook.

Example: Image search with CLIP

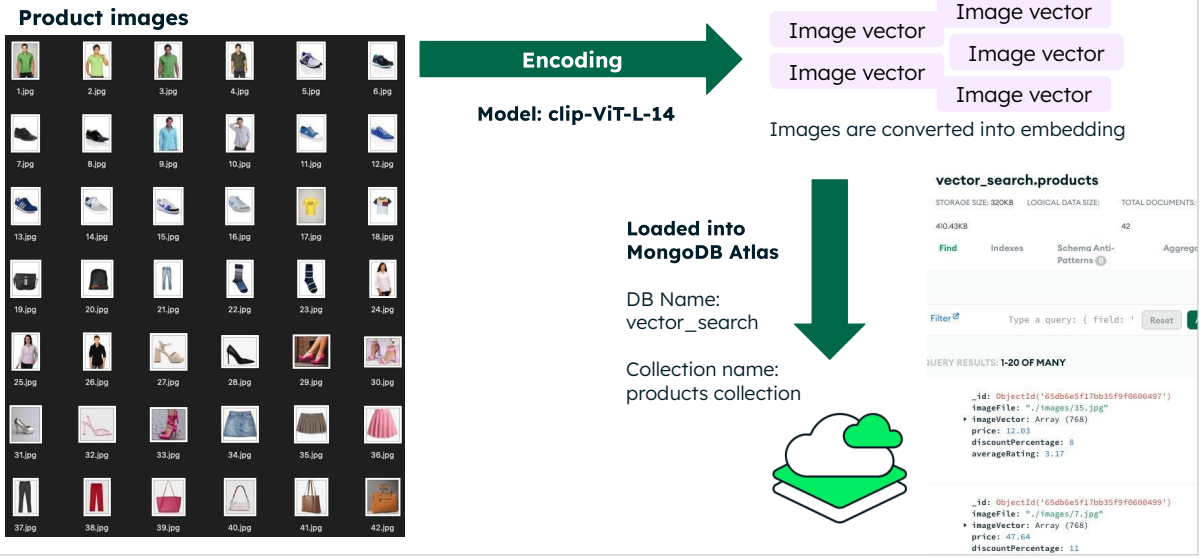


In this example we have a products collection, where the image of the product has been encoded into a vector.

The LLM is an image and text model, which maps images and text to the same vector space.

This means text searches can match images, rather than keywords.

Example: Image search with CLIP



In order to prepare this example, we have already converted a list of images into vectors and uploaded into a collection.
Download the Products collection, unzip the dump folder and restore it in your Atlas cluster (Note: you need at least 320KB to upload this collection)

Link and instructions are in the Student Workbook

Example: *vectorImg* index with filters

Create a new Atlas Vector Search index

Index name: ***vectorImg***

Collection: ***vector_search.products***

field: ***imageVector***

Additional filters:

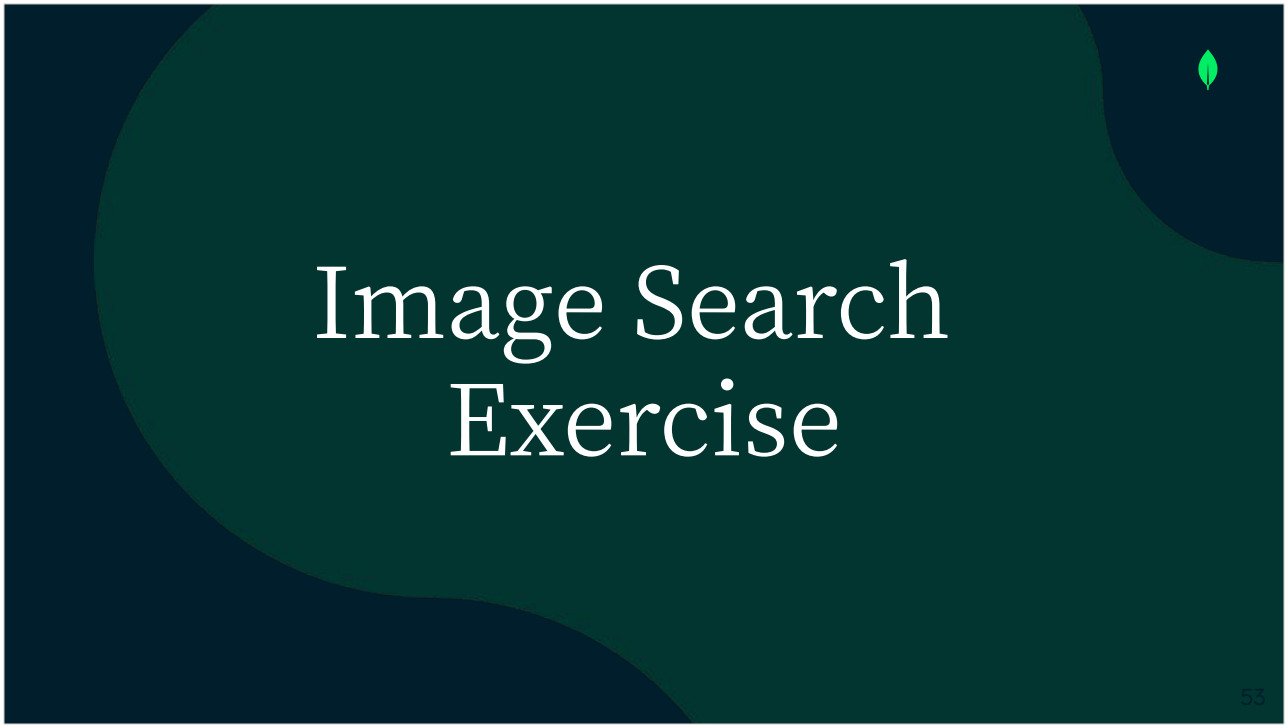
- ***price***
- ***averageRating***
- ***discountPercentage***

```
{
 "fields": [
   {
     "numDimensions": 768,
     "path": "imageVector",
     "similarity": "cosine",
     "type": "vector"
   },
   {
     "path": "price",
     "type": "filter"
   },
   {
     "path": "averageRating",
     "type": "filter"
   },
   {
     "path": "discountPercentage",
     "type": "filter"
   }
 ]
}
```


Note: If you are using Atlas M0, you will need to delete any of the previous indexes we created in this course.

M0 clusters are limited to a maximum of 3 Atlas Vector Search indexes.

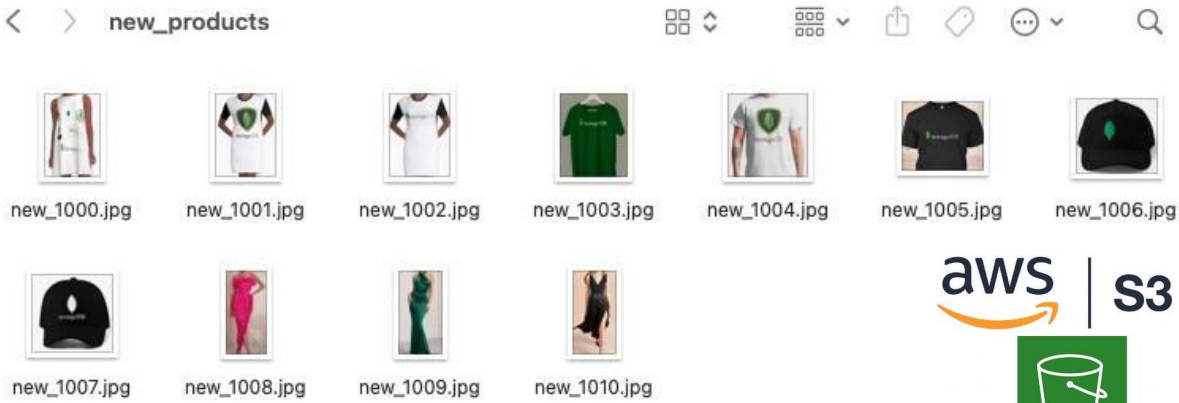
Make sure numDimensions matches the number of vector elements in each document.



Follow this exercise in the Student workbook.

Challenge: Image to Image search

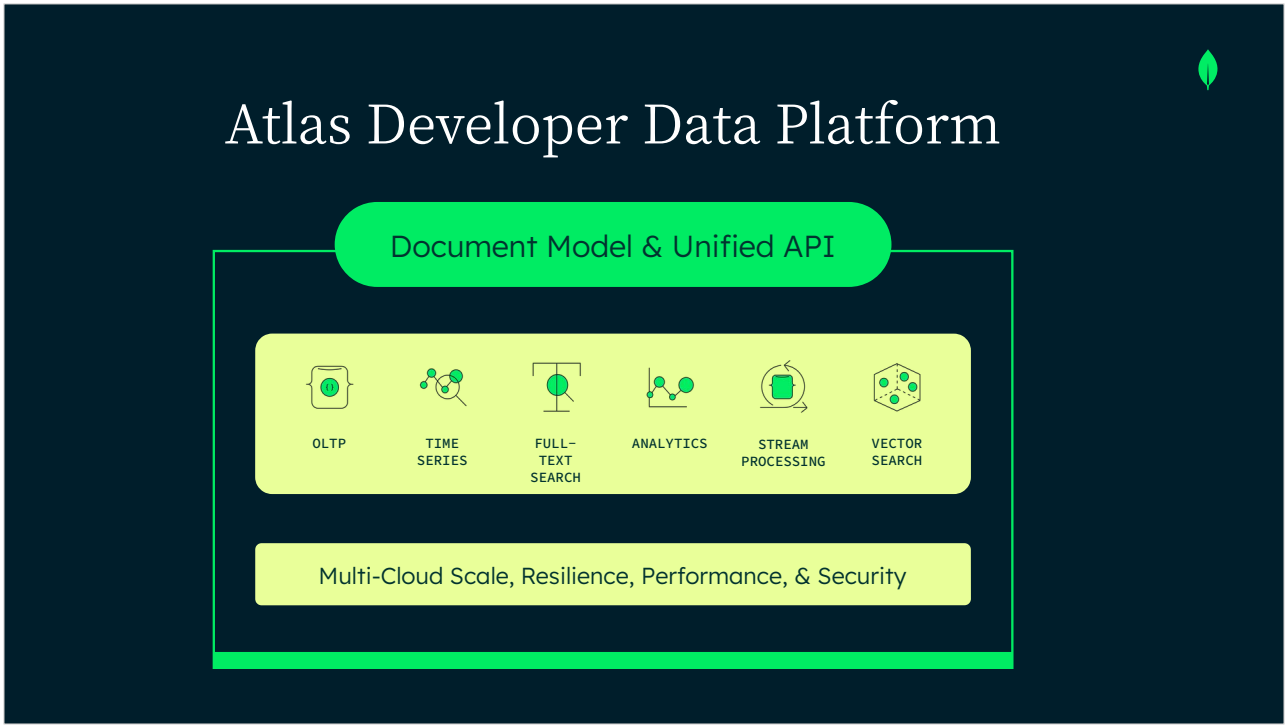
What happens when I want to find a similar product to another one?
I would like to use a image to search in the product collection for similar products



We would like to find products based on internet images or local images.
We will need to modify the Python code used in the Image Search exercise to include this functionality.
For Testing you final code, you can:

- Use the images in this AWS S3 bucket - Example:https://mdb-strigio.s3.eu-central-1.amazonaws.com/vectorsearch_new_products/new_1000.jpg
- Use images available on the internet - see this Example for a skirt
- Or even used the local images stored under encoder/images/ folder (those are exactly the same as the ones in the product collection)

Follow this exercise in the Student workbook.



Atlas platform offers much more than just Atlas Search and Vector Search.

- Atlas can provide a unified approach to building applications
- The developer data platform allows us to use more than just Atlas Search and Vector Search.
- Atlas Triggers can be used to encode any new documents added to a collection automatically.

Exercise - Part 1:
Setting Triggers to encode new documents

Create the Trigger:

Add a Trigger called *AddEmbeddingsTrigger* and configure the trigger settings:

Select your Cluster Name.

Database Name: sample_mflix

Collection Name: embedded_movies

Operation Type: Insert Document

Full Document: On

Replace the code with the provided Trigger code (next slide).

Trigger code

Your instructor will provide this code to you.

Replace <**API_KEY**> with your OpenAI API_KEY

```
exports = async function(changeEvent) {
  const _id = changeEvent.fullDocument._id;
  const plot = changeEvent.fullDocument.plot;


  // Get the MongoDB service you want to use (see "Linked Data Sources" tab)
  // Note: In Atlas Triggers, the service name is defaulted to the cluster name.
  const serviceName = "Cluster0";
  const database = "sample_mflix";
  var collName = "embedded_movies";

  // Get a collection from the context
  var collection = context.services.get(serviceName).db(database).collection(collName);


  var MDB_API_KEY = "<API_KEY>"
const arg = {
url: 'https://iltvectorsearch.openai.azure.com/openai/deployments/text-embedding-ada-002/embeddings?api-version=2023-05-15',
headers: {
  "Content-Type": ["application/json"],
  "api-key": [ MDB_API_KEY ]
},
body: JSON.stringify({
  input: plot,
})
  };


  const response = await context.http.post(arg);
  // The response body is a BSON binary object. Parse it to JSON.
  const responseBody = EJSON.parse(response.body.text());
  const embeddings = responseBody.data[0].embedding;


  try {
// Update the document with the new embeddings
const result = await collection.updateOne(
  { _id: _id },
  { $set: { plot_embedding: embeddings } }
);
// Access the number of documents modified
const modifiedCount = result.modifiedCount;
//console.log(`Modified ${modifiedCount} document(s)`);
return { result: modifiedCount };

  } catch (err) {
console.log("Error occurred while executing updateOne:", err.message);
return { error: err.message };
  }

};
```

Exercise - Part 2:
Setting Triggers to encode new documents

Now you can test this trigger by adding a new movie to the collection. Testing the results:

In your terminal, connect to the Atlas cluster

Type the following commands to insert one movie in the collection:

```
use sample_mflix
db.embedded_movies.insertOne({
     "title": "Barbie",
"plot": "Barbie and Ken are having the time of their lives in the colorful and seemingly perfect world of Barbie Land. However, when they get a chance to go to the real world, they soon discover the joys and perils of living among humans.",
     "imdb": {
           "rating": 6.9
          }})
```

Find the movie by searching the title “Barbie” and check the embedding was included.

```
db.embedded_movies.find({"title": "Barbie"})
```


Depending on the load on the OpenAI server, it may take a few seconds for the update to reflect.

MongoDB's applications as a Vector Database

Use of MongoDB as a Vector DB

Common Applications which leverage Vector Databases:

Large Language Models (LLMs)

Retrieval Augmented Generation (RAG)

LLMs and Semantic Search

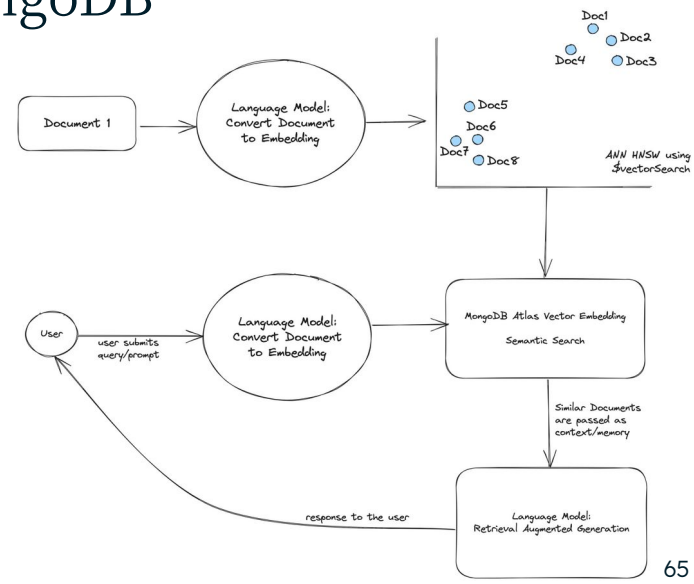
- So obviously Keyword search is useful when you're searching for something known in a article or table, but what about for all the other use cases?
- What if you're trying to search through a variety of rich media categories to find what you need, including videos, books, social media posts, PDFs, audio files and more? And how do you ensure you receive relevant results?
- Vector Search is a critical capability that addresses this need - **Searching and finding relevant results from unstructured data, based on semantic similarity**

Retrieval Augmented Generation (or RAG)

- RAG is about providing the context of proprietary data to engineer accurate, relevant responses for LLM-based applications. RAG enables these AI and generative AI use-cases.
- The need we've heard from our customers is augmenting LLM apps with their data, being able to add and index vectors to their existing collections without the need for changing their data model **or having to adopt a new tool.**

LLMs leverage MongoDB

- Contextual expansions
  Sentence completion
  Textual similarity and paraphrasing
  Domain-specific knowledge



The above diagram shows a system that uses LLMs and vector search to power semantic search. The system works as follows:

1. The user submits a query to the system.
2. The system uses an LLM to generate a vector representation of the query.
3. The system searches the vector search index for the vectors that are most similar to the query vector.
4. The system returns the documents that are associated with the most similar vectors to the user.

Embedding model vs. Language model

Embedding Model	Large Language Model
Converts text or unstructured data (video, audio, image, etc) into vector embeddings	Understands and generates natural language
Outputs Arrays of numbers (vector embeddings) - Length of the array is defined by the number of dimensions - The dimensionality depends on the embedding model (i.e, text-embedding-ada-002 model generates a vector with 1536 dimensions)	Outputs text, answers, translations, code, etc.
Examples: ***text-embedding-ada-002, clip-ViT-L-14***	Examples: ***GPT-3.5-turbo, Llama2, Gemini, Orca2, Mistral, Phi, etc***

In general, different models serve different purposes.
For example, OpenAI's GPT models are language models that understand and generate languages, the DALL-E model can create images based on a textual prompt, Whisper is a model that can convert audio into text, etc.

Retrieval Augmented Generation (RAG)

The retrieval-augmented generation (RAG) architecture was developed to address some of the current LLM limitations

- Hallucinations
- Stale data
- No access to users local data
- Token limit

RAG provides context to the LLM using retrieved documents from a initial vector search.

LLMs have revolutionized the field of natural language processing, but they do come with certain limitations:

- **Hallucinations**: LLMs sometimes generate factually inaccurate or ungrounded information, a phenomenon known as "hallucinations."
- **Stale data**: LLMs are trained on a static dataset that was current only up to a certain point in time. This means they might not have information about events or developments that occurred after their training data was collected.
- **No access to users' local data**: LLMs don't have access to a user's local data or personal databases. They can only generate responses based on the knowledge they were trained on, which can limit their ability to provide personalized or context-specific responses.
- **Token limits:** LLMs have a maximum limit on the number of tokens (pieces of text) they can process in a single interaction. Tokens in LLMs are the basic units of text that the models process and generate. They can represent individual characters, words, subwords, or even larger linguistic units. For example, the token limit for OpenAI's gpt-3.5-turbo is 4096.

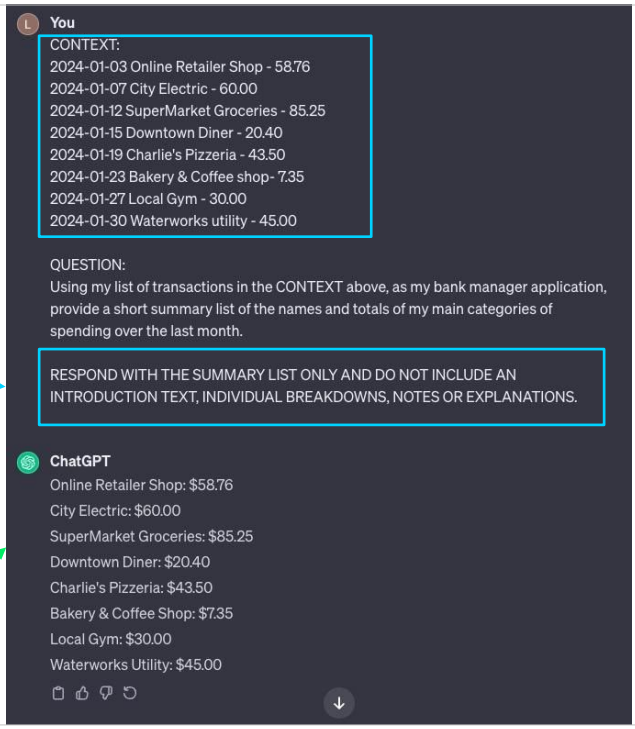
RAG uses vector search to retrieve relevant documents based on the input query. It then provides these retrieved documents as context to the LLM to help generate a more informed and accurate response. Instead of generating responses purely from patterns learned during training, RAG uses those relevant retrieved documents to help generate a more informed and accurate response.

- RAGs minimize hallucinations by grounding the model's responses in factual information.
- By retrieving information from up-to-date sources, RAG ensures that the model's responses reflect the most current and accurate information available.
- While RAG does not directly give LLMs access to a user's local data, it does allow them to utilize external databases or knowledge bases, which can be updated with user-specific information.
- Also, while RAG does not increase an LLM's token limit, it does make the model's use of tokens more efficient by retrieving only the most relevant documents for generating a response.

Why RAG?

The application retrieves the user's recent bank statement items from the database, injects this info (plus some control instructions) into the prompt and sends it to the LLM

Augmenting the question sending context to the LLM, generates a more accurate response



You can see inside the blue boxes, the additions the application makes to the prompt's text that it sends.
The application retrieves the user's recent bank transactions from the database and then augments the prompt with the historical context. This is a common pattern. An application is basically asking an LLM to make sense of the data it provides.

The application also does something else, which is common when engineering the prompts it sends. It injects some control instructions into the prompt to coerce the LLM to build its response in a specific format. For example, the app says to the LLM not to include any explanations in its response.

LLM now returns an accurate response relevant to the user's bank transaction history for the user which the application is asking on behalf of.

Why RAG?

RAG architecture is a framework that combines the benefits of both retrieval-based models and generation-based models in AI systems.

Several benefits:

- Better response diversity
- More reliable and accurate responses
- Personalization
- Efficient Knowledge utilization

This architecture is important for several reasons:

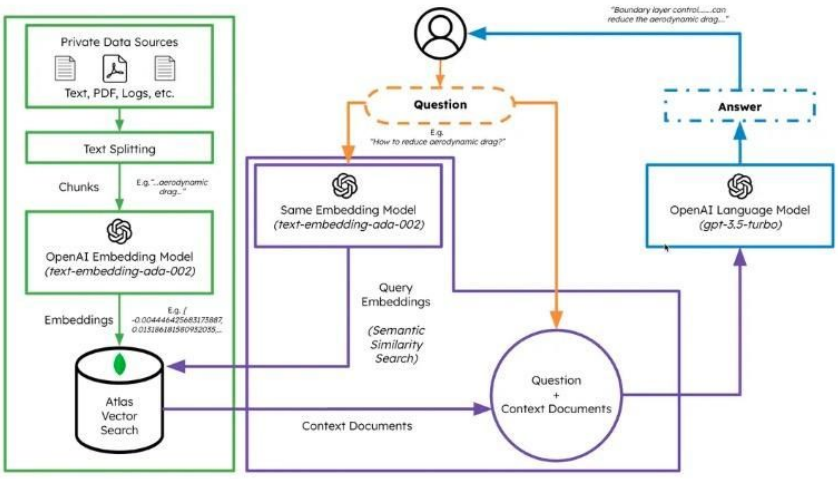
1. **Better response diversity:** By combining retrieval and generation, it allows for a wider range of responses, ensuring diverse and interesting output.

2. **More reliable and accurate responses:** The retrieval module helps ensure that the generated responses are based on reliable and relevant information, improving the accuracy and quality of the output.

3. **Personalization:** The retrieval module can be tailored to retrieve information specific to the user or context, allowing for personalized responses.

4. **Efficient knowledge utilization:** The retrieval module can efficiently search through a large knowledge base, saving computational resources and time compared to generating responses from scratch.

Example - RAG app architecture



70

Review this example using LangChain: https://www.mongodb.com/developer/products/atlas/rag-atlas-vector-search-langchain-openai/
LangChain is a framework designed to simplify the creation of LLM applications. It provides a standard interface for chains, lots of integrations with other tools, and end-to-end chains for common applications. This allows AI developers to build LLM applications that leverage external sources of data (for example, private data sources).

RAG Example Review

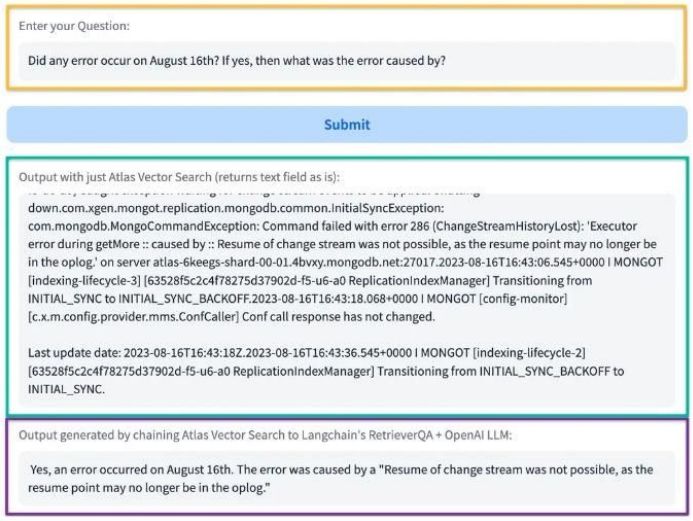
Review this code in the Github repository.
Review the following code: https://github.com/mongodb-developer/atlas-vector-search-rag
This example is not implemented during this course. It requires and OpenAI account.

Example RAG - Main components

LangChain	OpenAI	Atlas Vector Search	Gradio
DirectoryLoader: - All documents from a directory - Split and load - Uses the Unstructured package	Embedding Model: - text-embedding-ada-002 - Text → Vector embeddings - 1536 dimensions		UI for LLM app - Open-source Python library - Allows to quickly create user interfaces for ML models
RetrievalQA: - Retriever - Question-answering chain	Language model: - gpt-3.5-turbo - Understands and generates natural language - Generates text, answers, translations, etc.		
MongoDBAtlasVectorSearch: - Wrapper around Atlas Vector Search - Easily create and store embeddings in MongoDB collections - Search using Atlas Vector Search			

Based on the code example review, we can identify the following components

Example RAG - Use cases

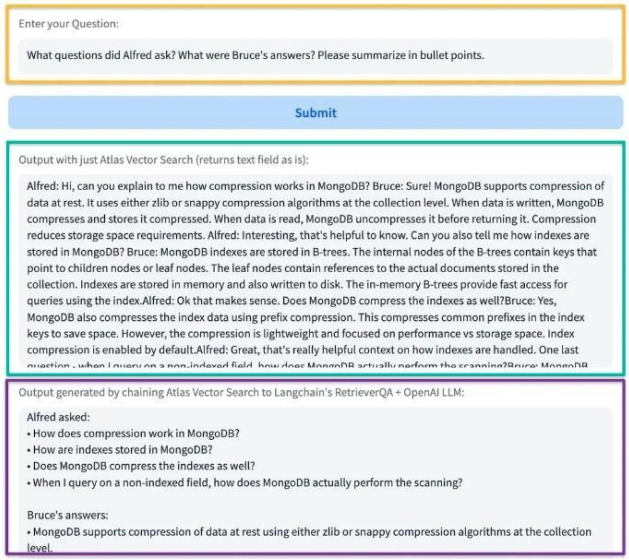


Data analysis based on our internal documents.

In this case we provide the MongoDB logs for context, then the LLM is able to answer specific questions about the system issues on specific time frames and dates.

This use case shows a Log analysis example.

Example RAG - Use cases

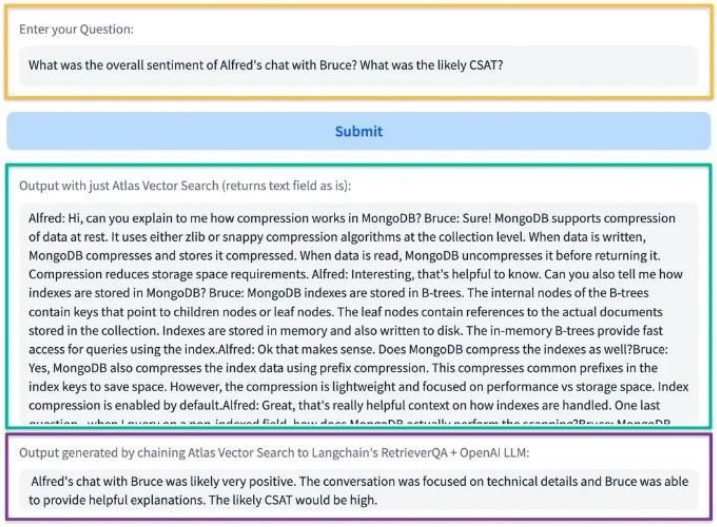


Chat / Meeting summarization based on call script, email thread, or chat.

In this case we provide chat conversation, email thread, or meeting transcript to the LLM.

This use case shows a synthesization and summarization example.

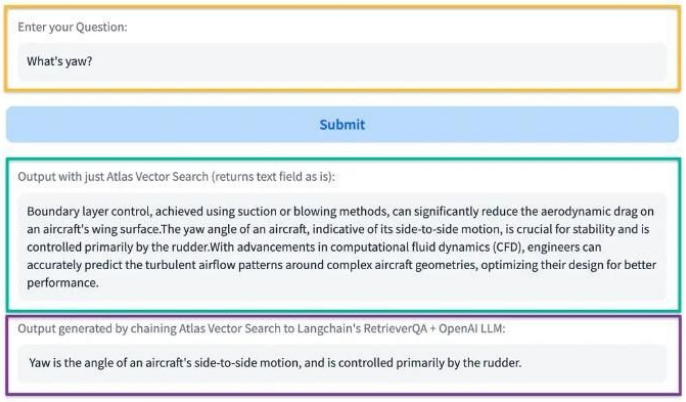
Example RAG - Use cases



Similar than the previous example:
We provide chat conversation, email thread, or meeting transcript to the LLM.

This use case shows a sentiment analysis example.
This could apply to customer feedback data, or products reviews.

Example RAG - Use cases



Precise answer retrieval:

We can provide product documentation, user manuals, training materials, etc.

This use case shows a Q&A tool application by retrieving a precise answer based on a set of documents and information.

Atlas Vector Search Nodes

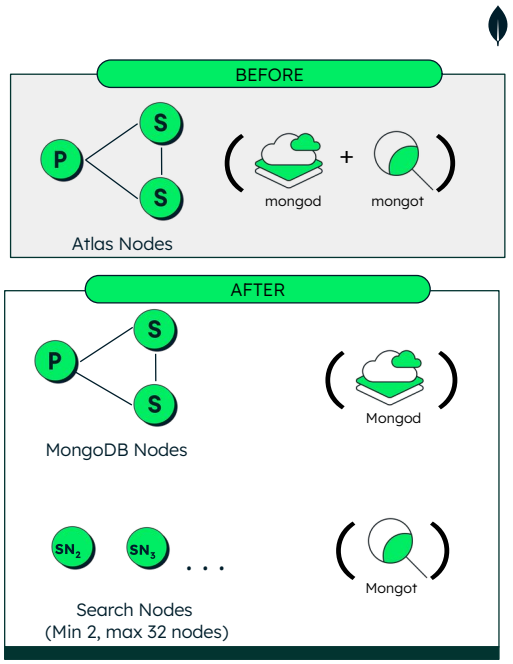
GA 7.x

Search Nodes on AWS

Resource Optimization and Flexibility
Confidently handle and scale demanding Search workloads

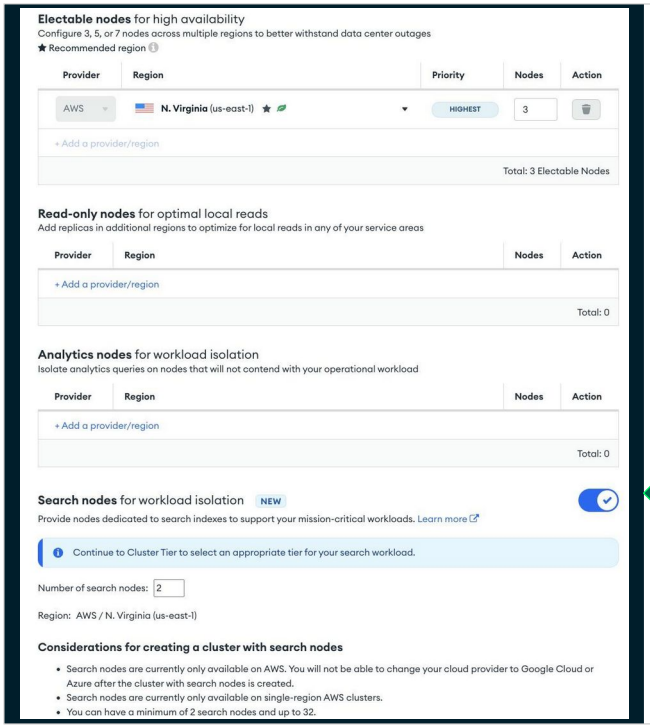
Better performance
40% - 60% decrease in query time for many complex queries

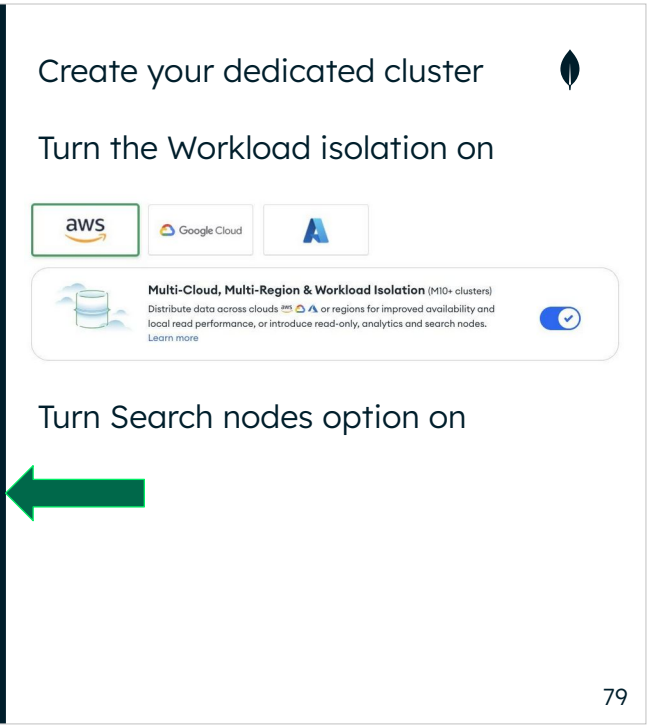
Higher Availability
Workload isolation avoids resource contention between database and search



78

For single-region AWS clusters, Atlas supports deploying search nodes for Atlas Search separately on AWS
(Google Cloud and Azure coming soon)





You can increase and reduce the number of search nodes that you deploy for your cluster when you create or modify a cluster.
You can deploy a minimum of 2 and a maximum of 32 search nodes on you Atlas cluster (charged hourly)
Atlas provides different search tiers. Each search tier has a default RAM capacity, storage capacity, and CPU.

Quiz Time!

#1. What is the main function of an embedding model?

A	Understand natural language	B	Generate natural language	C	Convert data into vector embeddings
D	Perform the vector similarity search	E	All of them		

Answer in the next slide.

#1. What is the main function of an embedding model?

A	Understand natural language	B	Generate natural language	C	Convert data into vector embeddings
D	Perform the vector similarity search	E	All of them		

#2. The audio_files collection includes audio files you encoded in a specific audio embedding model with vector embeddings of 768 items in the audio_embedding field.
This encoding model works best when looking for similarities on the vector angles (orientation). Which of the following statements are true?

A	The numDimensions field in the vector search index will be 768
B	The path in the vector search index is audio_files
C	The similarity function in the vector search index is euclidean
D	The similarity function in the vector search index is cosine
E	The similarity function in the vector search index is dotProduct

Answer in the next slide.

#2. The audio_files collection includes audio files you encoded in a specific audio embedding model with vector embeddings of 768 items in the audio_embedding field.
This encoding model works best when looking for similarities on the vector angles (orientation). Which of the following statements are true?

A	The numDimensions field in the vector search index will be 768	B	The path in the vector search index is audio_files	C	The similarity function in the vector search index is euclidean
D	The similarity function in the vector search index is cosine	E	The similarity function in the vector search index is dotProduct		

#3. In the $vectorSearch stage, which of these are parameters or fields we specify?

A	similarity function	B	limit	C	vectorSearchScore
D	path	E	numDimensions		

Answer in the next slide.

#3. In the $vectorSearch stage, which of these are parameters or fields we specify?

A	similarity function	B	limit	C	vectorSearchScore
D	path	E	numDimensions		

#4. What are the benefits of using the RAG architecture with MongoDB applications?

A	Minimize hallucinations by grounding the model's responses in factual information
B	Ensures that the model's responses reflect the most current and accurate information
C	MongoDB Vector Search creates the internal mathematical representation that identifies a token or word called embedding
D	Vector embeddings are integrated with application data and seamlessly indexed for semantic queries, enabling you to build easier and faster

Answer in the next slide.

#4. What are the benefits of using the RAG architecture with MongoDB applications?

A. Minimize hallucinations by grounding the model's responses in factual information

B. Ensures that the model's responses reflect the most current and accurate information

C. MongoDB Vector Search creates the internal mathematical representation that identifies a token or word called embedding

D. Vector embeddings are integrated with application data and seamlessly indexed for semantic queries, enabling you to build easier and faster

SKILL BADGE ASSESSMENT – 10 MINUTES

GenAI: Vector Search Fundamentals Questions

MongoDB Skill

Vector Search Fundamentals

Take the Assessment

https://mdb.link/Skill-VectorSearchFundamentals-PS

Ask your neighbor

Raise your hand

Let's do it together!

SKILL BADGE ASSESSMENT – 10 MINUTES

GenAI: RAG with MongoDB

MongoDB Skill

RAG with MongoDB

Take the Assessment

https://mdb.link/Skill-RAGMongoDB-PS

Questions

Ask your neighbor

Raise your hand

Let's do it together!

Recap

Vector embeddings allow unstructured data to be represented in a numerical format to allow comparison with other data.

In order to find similarities between we can use the Approximate Nearest Neighbors (ANN) technique. Hierarchical Navigable Small Worlds (HNSW) provides a fast method to find the approximate nearest neighbor.

Chunking is a computational method used in the field of AI and NLP that divides the text documents into meaningful units (words, sentences, paragraphs, etc).

Atlas Vector Search is an Atlas feature that allows us to create vector indexes and search for similar vectors to support Semantic search.

Main MongoDB's applications as a Vector Database are LLMS semantic search and RAG.

Appendix

Useful Resources

MongoDB Embedding Dataset

RAG with Gemma, MongoDB and Open Source Models

RAG System With LlamaIndex, OpenAI, and MDB Vector Database

Open source AI Cookbook